Tempore Language

Specifying and verifying when and how programs use resources

Boxed route **type**: departure from 2 s after the reservation, before 5 s

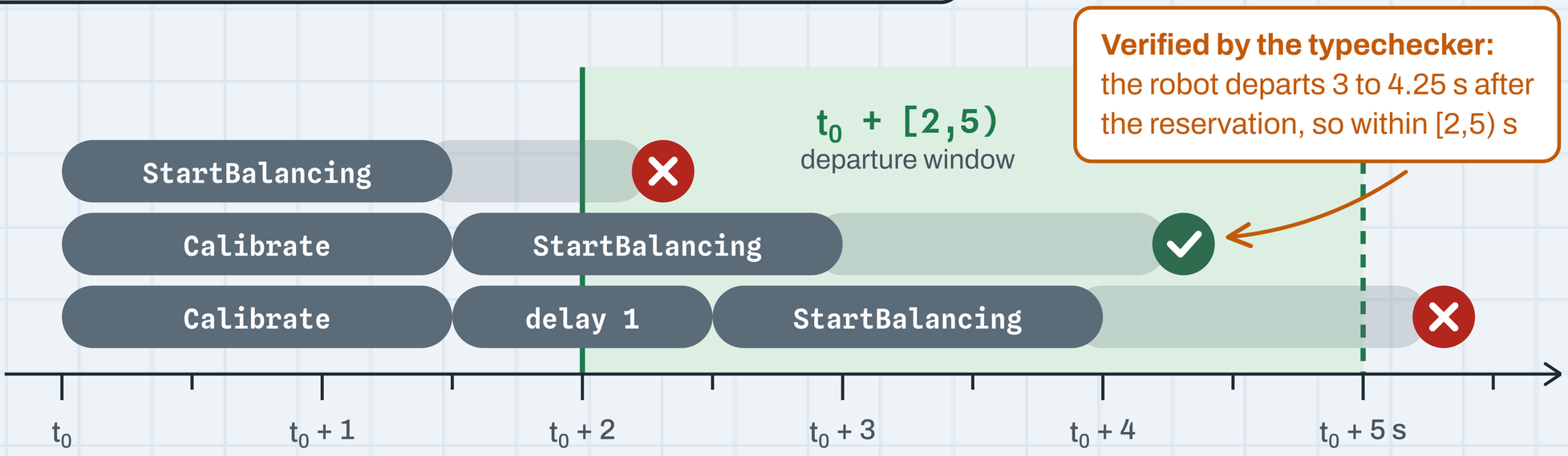
Effect grades: the running times of operations, in s

Default implementation for the interpreter; for the robot, the fleet manager reserves it

```
operation ReserveRoute    : position ~> [[2,5)]route # [1.0, 1.50)
operation Locate          : unit     ~>     position # (0.5, 1.00]
operation Calibrate       : unit     ~>         bias # [1.5, 2.00]
operation StartBalancing  : bias     ~>         unit # (1.5, 2.25]
```

```
default ReserveRoute here =
  delay 1;
  box [2,5) [here; (2.0, 1.0)]
```





Verified by the typechecker: the robot departs 3 to 4.25 s after the reservation, so within [2,5) s

EARLY.TPE — REJECTED

```
let here =
  perform Locate () in
let bias =
  perform Calibrate () in
let reserved =
  perform ReserveRoute here in
--- t0 ---
perform StartBalancing bias;
unbox reserved as route in
perform Drive route
```

Typing error: the step takes 1.5 to 2.25 s; the robot may depart too early

TIMELY.TPE — ACCEPTED

```
let here =
  perform Locate () in
let reserved =
  perform ReserveRoute here in
--- t0 ---
let bias =
  perform Calibrate () in
perform StartBalancing bias;
unbox reserved as route in
perform Drive route
```

Typechecks: the steps take 3 to 4.25 s; the robot departs in time

LATE.TPE — REJECTED

```
let here =
  perform Locate () in
let reserved =
  perform ReserveRoute here in
--- t0 ---
let bias =
  perform Calibrate () in
delay 1;
perform StartBalancing bias;
unbox reserved as route in
perform Drive route
```

Typing error: the steps take 4 to 5.25 s; the robot may depart too late

Beyond time: regular expressions, security levels, …

```
operation Estimate  : unit     ~> [{Scan*}]position # {Estimate}
operation Scan      : unit     ~>          distance # {Scan}
operation Turn      : angle    ~>              unit # {Turn}
operation PlanRoute : position ~>             route # {PlanRoute}
```

EXPLORE.TPE — REJECTED

```
let estimate = perform Estimate () in
let gap = perform Scan () in
(if gap < 20.0 then perform Turn 90.0);
unbox estimate as here in
perform PlanRoute here
```

Typing error: the position estimate may be used after the robot has already turned

Under the hood

Graded modal types: a resource of type $[\rho]A$ is usable at type A while the accumulated grade is within ρ

Graded effects and handlers: operations declare their effects as grades, which handlers must respect

Type and grade inference: types and grades are inferred automatically

Formal semantics: operational and denotational semantics; proven to be type-safe, sound and adequate

Tempore
Programs run using a small-step interpreter

ARDUINO · **M5STACK** CoreS3, BALA2, …
Compiled to C++ code for small embedded devices and robots

opentrons
Compiled to Python code for lab automation robots

Your platform could be here
Same grades and checks, securing your resource use

UNIVERSITY OF TARTU 1632

Danel Ahman · danel.ahman@ut.ee
Institute of Computer Science

tempore-lang.ee
github.com/danelahman/tempore-lang